10 goto 10

Hola Mundo\0

Manejo de archivos en C

Kalim Al Razif
Follow
Jul 19, 2015 · 5 min read

Si queremos que nuestros datos perduren en el tiempo no podemos confiar en la RAM, puesto que se borra cuando el fluido eléctrico se detiene. Para hacer que sean permanentes debemos guardarlos en un archivo.

Los archivos son ristras de datos que se guardan en un sistema de archivo dentro de un dispositivo de almacenamiento masivo; disco duro, pendrive, cdrom, dvdrom, cinta magnética, etc.

Los archivos pueden ser de cualquier tipo: fotos, música, películas, hojas de calculo, documentos de texto. Todo depende de que información deseemos guardada en ellos.

Si bien todos los archivos son información binaria, para los efectos de su uso en C reconoceremos dos tipos de archivos, los archivos de texto y los archivos binarios.

Abrir archivos

Para comenzar a usar un archivo dentro de nuestro programa debemos asignarlo a una variable, en el siguiente listado podemos ver como se hace.

```c
#include
<stdio.h>

        int main(){
          /*
           * Para usar un archivo tenemos que asociar una variable puntero
           * al archivo para poder escribir o leer de el
           */
          FILE *manejador_archivo = NULL;
          /*
           * Ahora que tenemos la variable para asignarla al archivo procedemos a
           * abrir el archivo, el primer parametro es el nombre del archivo
           * con su localizacion exacta en el sistema de archivos.
           * El segundo parametro es la forma en la que se va a abrir el archivo,
           * en este caso estamos abriendo el archivo en modo de solo lectura
           */
          manejador_archivo = fopen("/home/nomar/archivo", "r");

          /*
           * Ahora debemos comprobar que el proceso de apertura fue exitoso
           * para saber si nuestra apertura de archivo fue exitosa hacemos la
           * siguiente comparacion, si el contenido de la variable es igual a NULL
           * el archivo no fue abierto.
           */
           if(manejador_archivo == NULL ) {
             printf("No fue posible abrir el archivo\n");
             return -1;
           }

          /*
           * Cuando ya no necesitemos trabajar mas con el archivo procedemos a cerrarlo
           */
          fclose(manejador_archivo);

          return 0;
        }
```

view rawabrir_archivo.c hosted with ♡ by **GitHub**

En la línea 8 encuentra la declaración de la variable que usaremos para manipular el archivo, es una variable puntero de tipo *FILE.*

Haciendo uso de la instrucción *fopen()* le pedimos al sistema operativo que nos localice el archivo nombrado en el primer parámetro, y que lo abra en el modo solicitado en el segundo parámetro. Este archivo esta ubicado en un sistema de archivos ***NIX**, si el sistema de archivo fuese de Windows el nombre de archivo comenzaría con una unidad, por ejemplo *"C:**archivo"*.

Modos de apertura de un archivo

El segundo parámetro de *fopen()* indica la forma en que se manipulara el archivo. En el ejemplo anterior en la linea 16 estamos abriendo el archivo para solo leerlo. ¿Que quiere decir? Que el archivo debe existir para poderlo abrir y que no se le añadirá o modificara el contenido actual del archivo.

Las formas de abrir el archivo son estas:

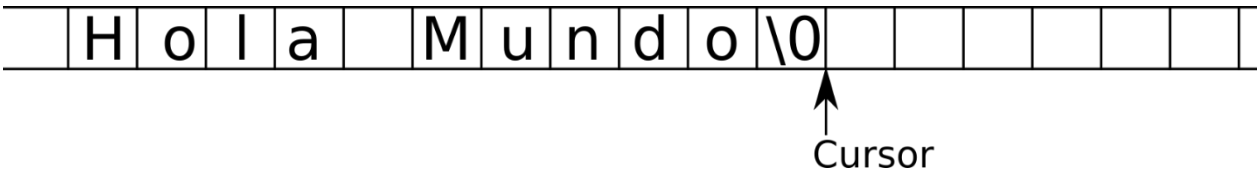
- r — abre el archivo en modo de solo lectura.
- w — abre el archivo para escritura (si no existe lo crea, si existe lo destruye).
- a — abre el archivo para agregar información (si no existe lo crea).
- r+ — abre el archivo para lectura/escritura (comienza al principio del archivo).
- w+ — abre el archivo para lectura/escritura, sobre-escribe el archivo si este ya existe o lo crea si no).
- a+ — abre el archivo para lectura/escritura (se sitúa al final del archivo).

Si vamos a trabajar con archivos binarios usamos la letra *b* así que los modos de acceso quedan “rb”, “wb”, “ab”, “rb+”, “wb+”, “ab+”.

Cerrar archivos

Luego de trabajar con un archivo lo recomendable es cerrarlo, esto hace que el sistema operativo escriba cualquier dato que pudiera estar en memoria a disco. Como se puede ver en el ejemplo, en la linea **32** la instrucción *fclose()* cierra el archivo. Recibe un parámetro, que es, la variable con la que se maneja el archivo.

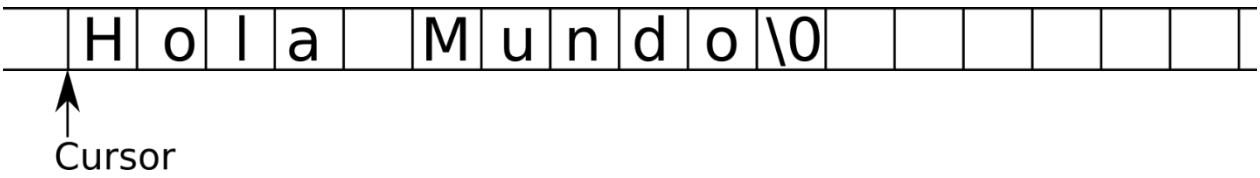
Rebobinar archivos



El cursor de archivo indica en que sitio del archivo estamos ubicados.

Cuando se trabaja con archivos se cuenta con lo que se llama un cursor, un número que indica la posición del archivo en la que nos encontramos. La instrucción *rewind()* regresa el cursor al principio del archivo. Recibe un parámetro, la variable con la que se maneja el archivo.

El cursor de archivo indica el punto a partir del cual se leerán o escribirán datos.



Luego de un rewind() el cursor quedara al principio del archivo.

Moverse en el archivo

Es posible mover el cursor de archivo haciendo uso de la instrucción *fseek()* que acepta 3 parámetros. El primer parámetro es la variable de archivo, el segundo parámetro es la posición en bytes, y el tercero es uno de estos tres:

- **SEEK_SET** cuenta la posición a partir del principio del archivo.
- **SEEK_CURRENT** cuenta la posición a partir de la posición actual del cursor.
- **SEEK_END** cuenta la posición desde el final del archivo.

```
#include
<stdio.h>

        int main(){
          // Variable de archivo
          FILE *datos = NULL;
          // Abrimos el archivo
          datos = fopen("/home/nomar/datos", "a");

          // Comprobamos que de verdad abrio
           if(manejador_archivo == NULL ) {
             printf("No fue posible abrir el archivo\n");
             return -1;
           }

          // Rebobinamos el archivo y llevamos el cursor al principio.
          rewind(datos);

          // Ahora con fseek nos posicionamos donde querramos,
          // en este caso 300 bytes desde el inicio del archivo.
          fseek(datos, 300, SEEK_SET);

          // Se ubica a 200 bytes del final del archivo
          fseek(datos, 200, SEEK_END);

          // Se mueve el cursor 3 bytes a partir de la posicion anterior
```

```
            fseek(datos, 3, SEEK_CURRENT);

            // Al final cerramos
            fclose(manejador_archivo);

            return 0;
        }
```

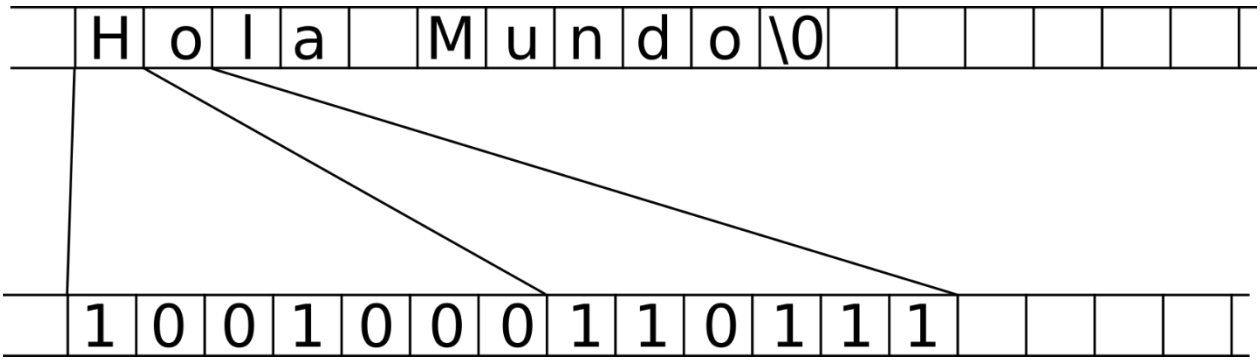
archivo_rewind_fseek.c hosted with ♡ by GitHub

¿Estoy al final del archivo?

¿Como saber si estamos al final de un archivo? La instrucción *feof()* nos retorna **1** o **true** si estamos efectivamente al final del archivo. Recibe un solo parámetro, la variable del archivo.

Archivos de texto

Los archivos de texto son archivos que están conformados por caracteres de texto plano (ASCII, UTF-8, o algún otro tipo de codificación de texto). Estos archivos podemos editarlos con un editor de textos sencillo como notepad en Windows o gedit en Linux.



Los archivos de texto al fin y al cabo son también archivos binarios, pero los tratamos como un tipo de archivo distinto, porque los podemos ver y editar desde un editor de texto.

Leer del archivo de texto

Para leer datos desde un archivo de texto contamos con las siguientes funciones:

- *fgetc()* permite leer un carácter desde el archivo, recibe un parámetro y es la variable del archivo. Devuelve el carácter leído.
- *fgets()* lee cadenas completas desde el archivo, hasta que encuentra un retorno de carro **\n** o un **eof**. Acepta tres parámetros el nombre de el arreglo de caracteres donde se va a guardar lo leído, cantidad de bytes a leer y variable de archivo.
- *fscanf()* funciona de la misma forma que un *scanf()* tradicional pero con un parámetro al principio que indica el archivo a leer.

```
/*
   * Suponga que el contenido del archivo es:
   * Gol
   * Hola mundo
   * Esto es un numero con formato  3.141
   * 3.141
   *
   * Con esto en mente, empecemos.
   */
   #include <stdio.h>

   int main(){
     // Variable archivo.
     FILE *entrada;
     // Variable numerica.
     float numero = 0.0;
     // Arreglo de caraceres para guardar cadenas.
     char buffer[100] = "";
     // Variable para guardar una letra.
```

```
    char letra;
    // Variable para los ciclos
    int i;

   entrada = fopen("./archivo.txt", "r");

   // Si no podemos abrir el archivo, terminamos el programa.
   if(entrada == NULL) { printf("No se pudo abrir el archivo... \n"); return -1; }

   // Con este ciclo leemos la primera linea Gol con su respectivo
   // retorno de carro.
   for(i = 0; i<4;i++){
     letra = fgetc(entrada);
   }

   // El resultado de usar fgets sera que en buffer quedara guardada
   // la linea Hola mundo con su respectivo retorno de carro.
   fgets(buffer,100,entrada);

   // Ahora leemos Esto es un numero con formato  3.141
   fgets(buffer, 100, entrada);

   // Con fscanf leemos el numero (caracteres) y lo guardamos como una variable flotante.
   fscanf(entrada, "%f", &numero);

   fclose(entrada);

   return(0);
  }
```

archivos_texto2.c hosted with ♡ by **GitHub**

Escribir en el archivo de texto

- *fputc()* escribe un carácter en el archivo, recibe dos parámetros, el carácter a escribir y la variable de archivo.

- *fputs()* escribe una cadena en el archivo, recibe dos parámetros, la cadena a escribir, y la variable de archivo.

- *fprintf()* funciona de la misma forma que *printf()* pero su primer parámetro es la variable de archivo.

```
#include
<stdio.h>

        int main(){
          // La variable de archivo
          FILE *archivito;
          // Variable para escribir datos en el archivo
          char buffer[100] = "Hola Mundo";

          // Variable numerica
          float numero = 3.1416;

          archivito = fopen("./archivo.txt", "a+");

          // Si no podemos abrir el archivo, terminamos el programa
          if(archivito == NULL) { printf("No se pudo abrir el archivo... \n"); return -1; }

          fputc('G', archivito);
          fputc('o', archivito);
          fputc('l', archivito);
          fputc('\n', archivito);
          // La primera linea del archivo fue escrita y contiene la frase Gol\n

          // Intentemos escribir algo mas, una cadena!
          // Pero ojo, solo esta escribiendo Hola Mundo, falta el retorno de carro
          // para completar la linea.
          fputs(buffer, archivito);

          // Ahora le toca el turno a fprintf
          fprintf ("\nEsto es un numero con formato %3.3f\n%3.3f", numero,numero);

          fclose(archivito);

          return 0;
        }
```

archivos_texto1.c hosted with ♡ by **GitHub**

Archivos binarios

Las anteriores funciones sirven para manejar archivos donde se guardan caracteres. Es decir si queremos guardar el numero 123456 ocupara 6 bytes, o 12 bytes dependiendo de la codificación que estemos usando. En ocasiones esto no es deseable. Queremos guardar la mayor cantidad de información ocupando la menor cantidad de espacio. Usando un formato binario nuestro numero 123456 ocupara ahora 4 bytes (lo que mide un *int* en C).

Cuando trabajamos en formato binario muy a menudo trabajamos con **estructuras**, que nos permiten empaquetar un conjunto de datos y tratarlo como si fuese una unidad.

Para escribir en archivos binarios

- *fwrite()* se usa para escribir datos binarios en un archivo, recibe tres parámetros, el primero es la variable a guardar, el segundo el tamaño de la variable a guardar, el tercero cuantas veces se va a guardar y por ultimo la variable de archivo.

```
#include
<stdio.h>

        // Esta estructura se declara global para que todas las funciones del programa
        // tengan acceso a su definición.
        struct formulario {
          char nombres[200];
          char apellidos[200];
          short edad;
        };

        int main(){
          // Declaramos una variable llamada persona, del tipo struct formulario
          struct formulario persona;

          // Variable de archivo
```

```
  FILE *salida;
  // Procedemos a rellenar la variable
  strcpy(persona.nombres,"Nomar Oscar");
  strcpy(persona.apellidos, "Mora Tovar");
  persona.edad = 40;

  // Abrimos el archivo notese la b para indicar que es binario
  salida = fopen("./datos.dat", "wb");

  // Si no podemos abrir el archivo, terminamos el programa.
  if(salida == NULL) { printf("No se pudo abrir el archivo... \n"); return -1; }

  // Escribimos 1 registro de tipo struct formulario
  fwrite(&persona, sizeof(struct formulario), 1, salida);

  return(0);
}
```

archivos_binarios1.c hosted with ♡ by **GitHub**

Para leer de un archivo binario

- fread() se usa para leer datos binarios de un archivo, recibe tres parámetros, el primero es la variable donde se van a guardar los datos, el segundo el tamaño de la variable a leer, el tercero cuantas veces se va a leer y por último la variable de archivo.

```
#include
<stdio.h>

// Esta estructura se declara global para que todas las funciones del programa
// tengan acceso a su definición.
struct formulario {
  char nombres[200];
  char apellidos[200];
  short edad;
};

int main(){
  // Declaramos una variable llamada persona, del tipo struct formulario
```

```
        struct formulario persona;

        // Variable de archivo
        FILE *salida;

        // Abrimos el archivo notese la b para indicar que es binario
        salida = fopen("./datos.dat", "rb");

        // Si no podemos abrir el archivo, terminamos el programa.
        if(salida == NULL) { printf("No se pudo abrir el archivo... \n"); return -1; }

        // Escribimos 1 registro de tipo struct formulario
        fread(&persona, sizeof(struct formulario), 1, salida);

        printf("El nombre es: %s %s", persona.nombres, persona.apellidos);
        printf("La edad es: %d", persona.edad);

        return(0);
      }
```

archivos_binarios2.c hosted with ♡ by **GitHub**

Material de apoyo

En el siguiente enlace podrán encontrar un proyecto de anjuta con ejemplos para cada proceso.

* * *